

AI Vibration Intelligence Platform

Stage 1 — Equipment Master Data

Complete Technical Documentation

Backend · Frontend · Database · APIs · Workflow
Architecture · File-by-File Breakdown · Connection Map

Generated: June 2026 | Stack: FastAPI + React + PostgreSQL

Table of Contents

1.	Project Overview & Architecture
2.	Technology Stack
3.	Backend — File-by-File Documentation
3.1	app/main.py
3.2	app/config.py
3.3	app/database.py
3.4	app/models/equipment.py
3.5	app/models/sensor.py
3.6	app/schemas/equipment.py
3.7	app/crud/equipment.py
3.8	app/routers/equipment.py
3.9	app/routers/lookups.py
3.10	alembic/ — Database Migrations
4.	Frontend — File-by-File Documentation
4.1	src/main.tsx & index.html
4.2	src/App.tsx
4.3	src/api/client.ts
4.4	src/api/equipment.ts
4.5	src/types/equipment.ts
4.6	src/pages/Dashboard.tsx
4.7	src/pages/EquipmentMaster.tsx
4.8	src/components/equipment/EquipmentForm.tsx
4.9	src/components/equipment/tabs/ (all 6 tabs)
4.10	src/components/equipment/SensorMountingDiagram.tsx
4.11	src/components/ui/ (FormField, MultiSelect, SectionCard, Toast)
4.12	src/lib/utils.ts
5.	Database — Schema & Design
6.	API Reference — All Endpoints
7.	Frontend ↔ Backend Connection Map
8.	Dashboard Image — What You See & Why
9.	AI Readiness Score — Calculation & Logic
10.	Full Request/Response Workflow
11.	Deployment & Infrastructure

1. Project Overview & Architecture

The **AI Vibration Intelligence Platform** is an industrial IoT / predictive maintenance web application. Stage 1, documented here, is the **Equipment Master Data** module — a system that allows engineers to register every rotating machine in a plant with full metadata: location hierarchy, mechanical specs, rotating component parameters, operating conditions, sensor configurations, and AI readiness flags. This rich metadata becomes the foundation for future stages (real-time vibration data ingestion, AI-based fault detection, alarm management, and predictive maintenance scheduling).

Layer	Technology	Purpose
Frontend	React 18 + TypeScript + Vite	Browser SPA — 2 pages, 6-tab form, equipment list
API Layer	FastAPI (Python 3.11)	RESTful JSON API, image upload, validation, CORS
ORM	SQLAlchemy 2.0 + Alembic	Database models, queries, schema migrations
Database	PostgreSQL 16 (Docker)	Persistent storage of all equipment & sensor data
State Mgmt	TanStack React Query v5	Server state, caching, background refetch
Forms	React Hook Form + Zod	Multi-tab form state, schema validation
Styling	Tailwind CSS 3.4	Utility-first responsive CSS

Architecture Flow

Browser → Vite Dev Server (port 5173)

The React SPA runs in the browser. All API calls are made via Axios to the FastAPI backend.

FastAPI Backend (port 8000)

Receives HTTP requests, validates them through Pydantic schemas, executes CRUD operations via SQLAlchemy, and returns JSON responses. Image uploads are stored on disk in *backend/uploads/*.

PostgreSQL Database (port 5433 on host, 5432 in container)

Docker container running PostgreSQL 16. SQLAlchemy connects via connection string in *.env*. Two tables: **equipment_masters** and **sensor_configurations**.

2. Technology Stack

Backend Dependencies (requirements.txt)

fastapi==0.115.0	Web framework — routing, dependency injection, OpenAPI docs auto-generation
uvicorn[standard]	ASGI server that runs FastAPI — handles HTTP/WebSocket connections
sqlalchemy==2.0.35	ORM — maps Python classes to PostgreSQL tables, handles all queries
alembic==1.13.3	Database migration tool — version-controls schema changes
psycopg2-binary	PostgreSQL driver — the actual connection adapter between SQLAlchemy and Postgres
python-multipart	Required for FastAPI to handle multipart/form-data (file uploads)
pydantic==2.9.2	Data validation and serialization — defines request/response schemas
pydantic-settings	Loads config from .env file into Settings class
pillow==10.4.0	Image processing library (imported, used for potential future image ops)
aiofiles==24.1.0	Async file I/O (available for async image handling)

Frontend Dependencies (package.json)

react@18.3.1	UI library — component tree, virtual DOM, hooks
react-router-dom@6.27	Client-side routing — / /equipment/new /equipment/:id/edit
@tanstack/react-query@5	Server state management — fetching, caching, background sync
react-hook-form@7.53	Form state management — tracks field values, errors, dirty state
zod@3.23	TypeScript-first schema validation — validates form data before API call
@hookform/resolvers	Bridge between Zod schema and React Hook Form
axios@1.7.7	HTTP client — makes all API calls to FastAPI backend
tailwindcss@3.4	Utility CSS framework — all styling done via class names
lucide-react@0.454	Icon library — all icons in the UI (Cpu, Trash2, Edit2, etc.)
@radix-ui/*	Accessible headless UI primitives — Dialog, Select, Toast, Popover
vite@5.4	Build tool and dev server — instant HMR, fast TypeScript bundling
typescript@5.6	Type safety across all frontend files

3. Backend — File-by-File Documentation

3.1 backend/app/main.py

Role: The application entry point. This file creates the FastAPI app instance, registers all middleware, mounts routers, and sets up the static file directory for uploaded images.

FastAPI()	Creates the ASGI application with title, description, and version metadata. These appear in auto-generated /docs (Swagger UI)
CORSMiddleware	Allows the React frontend (http://localhost:5173) to make cross-origin requests. Without this, browsers would block all API calls
app.include_router()	Registers the equipment router (prefix: /api/v1/equipment) and lookups router (prefix: /api/v1/lookups)
UPLOAD_DIR	Reads UPLOAD_DIR from environment variable (defaults to 'uploads'). Creates the directory if it doesn't exist
GET /health	Health check endpoint — returns {status: ok}. Used to verify the backend is running

3.2 backend/app/config.py

Role: Centralized configuration management using **pydantic-settings**. Reads all environment variables from the .env file and exposes them as a typed settings singleton used throughout the application.

database_url	PostgreSQL connection string, e.g. postgresql://user:pass@localhost:5433/dbname
secret_key	Application secret (currently unused — reserved for JWT authentication in future stages)
upload_dir	Directory path where equipment images are stored (default: 'uploads')
max_image_size_mb	Maximum allowed image upload size in MB (default: 10)
class Config	Tells pydantic-settings to load from .env file and ignore extra keys

3.3 backend/app/database.py

Role: Sets up the SQLAlchemy database engine, session factory, and base class for all ORM models. Also provides the **get_db()** dependency injected into every API endpoint.

create_engine()	Creates the SQLAlchemy engine with the PostgreSQL connection string. pool_pre_ping=True tests the connection before use to handle dropped connections
SessionLocal	Session factory — each call creates a new database session (transaction). autocommit=False means changes must be committed explicitly
Base	declarative_base() — the parent class all ORM model classes inherit from. SQLAlchemy uses it to track the table registry

<code>get_db()</code>	FastAPI dependency function. Yields a database session and ensures it's closed after the request completes (even on error). Every router endpoint that needs DB access uses <code>Depends(get_db)</code>
-----------------------	--

3.4 backend/app/models/equipment.py

Role: Defines the **Equipment** ORM model — the Python class that maps to the **equipment_masters** PostgreSQL table. Every column in the database is declared here.

id (UUID)	Primary key — auto-generated UUID4. Stored as PostgreSQL native UUID type
plant_name / area / line	Three-level location hierarchy. All required (NOT NULL). Organises machines by factory → zone → production line
machine_name	Human-readable name (e.g. 'Cooling Water Pump P-204'). Required
machine_id	Unique asset code (e.g. 'PUMP-P204'). Has UNIQUE constraint + index for fast lookup
machine_type	Category: Motor, Pump, Fan, Blower, Compressor, Gearbox, Turbine, Generator, etc.
machine_criticality	Four-tier: Low / Medium / High / Critical
rated_power_kw	Numeric(10,2) — e.g. 75.50 kW
rated_rpm	Integer — nameplate speed, e.g. 1480
bearing_number_de / nde	Drive End and Non-Drive End bearing catalogue numbers — used for fault frequency calculations
operating_speed_min/max	Actual operating speed range. Used in AI readiness check
operating_environment	PostgreSQL ARRAY(String) — multi-value field, e.g. ['Dusty', 'High Temperature']
equipment_image_path	Relative file path to the uploaded image, e.g. 'uploads/uuid.png'
machine_train_configured	Boolean AI readiness flag — has the machine chain been defined?
bearing_database_mapped	Boolean AI readiness flag — have bearing numbers been entered?
operating_mode_configured	Boolean AI readiness flag — have speed and load ranges been set?
asset_status	Current state: Active / Inactive / Under Maintenance / Decommissioned
created_at / updated_at	Auto-managed timestamps. <code>updated_at</code> uses <code>onupdate=datetime.utcnow</code>
sensors (relationship)	SQLAlchemy ORM relationship to <code>SensorConfiguration</code> . <code>cascade='all, delete-orphan'</code> means deleting equipment auto-deletes its sensors

3.5 backend/app/models/sensor.py

Role: Defines the **SensorConfiguration** ORM model — maps to the **sensor_configurations** table. Each sensor belongs to one piece of equipment (many-to-one).

id	UUID primary key, auto-generated
-----------	----------------------------------

equipment_id	Foreign key → equipment_masters.id with ondelete=CASCADE
sensor_type	Category: IEPE Accelerometer, Velocity Sensor, Temperature, etc.
mounting_location	Where physically mounted: Bearing Housing DE/NDE, Motor DE/NDE, Gearbox, etc.
orientation	Measurement direction: Horizontal / Vertical / Axial / Radial / Tangential
mounting_method	How attached: Stud Mounted, Magnetic Base, Adhesive, etc.
sensitivity	Numeric(10,4) — sensor output per unit, e.g. 100.0 mV/g
sensitivity_unit	Unit: mV/g, mV/mm/s, mV/μm, mA, V
sampling_rate	Pre-set string (e.g. '4096 Hz') or 'Custom'
sampling_rate_custom	Integer Hz value when sampling_rate = 'Custom'
frequency_range	Pre-set string (e.g. '0-5000 Hz') or 'Custom'
frequency_range_custom_min/max	Integer bounds when frequency_range = 'Custom'
is_active	Boolean — can deactivate a sensor without deleting it

3.6 backend/app/schemas/equipment.py

Role: Pydantic schemas define the shape of data coming **into** and going **out of** the API. They perform automatic validation — FastAPI rejects invalid requests with a 422 error before they reach the CRUD layer. There are separate schemas for Create, Update, and Output.

SensorConfigBase	Shared fields for all sensor schemas. Defines defaults and optional fields
SensorConfigCreate	Inherits SensorConfigBase — used when POST /sensors is called
SensorConfigUpdate	All fields Optional — allows partial updates via PUT
SensorConfigOut	Adds id, equipment_id, created_at. from_attributes=True lets it read from ORM objects
EquipmentBase	All equipment fields with defaults. Shared by Create and Out schemas
EquipmentCreate	Extends EquipmentBase, adds sensors: List[SensorConfigCreate]
EquipmentUpdate	All fields Optional — enables PATCH semantics (only send changed fields)
EquipmentOut	Full equipment response including id, image path, sensor list, timestamps
EquipmentListItem	Lightweight schema for the dashboard list — omits heavy fields like bearing_details, maintenance_notes
AIReadinessOut	Response for /ai-readiness endpoint: score_percent + 5 boolean checks
PaginatedEquipment	Wraps list response: total, page, page_size, items[]

3.7 backend/app/crud/equipment.py

Role: The data access layer. All database operations are here — completely separated from routing logic. Each function receives a SQLAlchemy Session and returns ORM objects or primitives.

<code>get_equipment_list()</code>	Queries with optional filters (<code>plant_name ilike</code> , <code>machine_type ==</code> , <code>criticality ==</code>). Applies ORDER BY <code>created_at DESC</code> , then LIMIT/OFFSET pagination
<code>get_equipment_by_id()</code>	Simple SELECT WHERE <code>id = ?</code> — returns None if not found
<code>get_equipment_by_machine_id()</code>	Used to check for duplicate machine IDs before creating
<code>create_equipment()</code>	Extracts sensors from payload, creates Equipment first (<code>db.flush()</code> to get the id), then creates each SensorConfiguration with the new <code>equipment_id</code> . Commits atomically
<code>update_equipment()</code>	Loads equipment, iterates <code>model_dump(exclude_unset=True)</code> so only provided fields are changed, commits
<code>delete_equipment()</code>	Loads and deletes — cascade handles sensors automatically
<code>update_image_path()</code>	Sets <code>equipment_image_path</code> to file path string (or None to remove)
<code>get_sensors_by_equipment()</code>	Returns all sensors WHERE <code>equipment_id = ?</code>
<code>create_sensor() / update_sensor() / delete_sensor()</code>	Sensor-level CRUD — same pattern as equipment
<code>compute_ai_readiness()</code>	Pure function — no DB call. Takes an equipment ORM object and evaluates 5 boolean checks (see Section 9 for full logic)

3.8 backend/app/routers/equipment.py

Role: Defines all HTTP endpoints under `/api/v1/equipment`. Each endpoint validates the request using Pydantic schemas, calls the appropriate CRUD function, and returns a typed response. FastAPI handles serialization automatically.

Method	Endpoint	Description
POST	<code>/api/v1/equipment/</code>	Create new equipment. Checks for duplicate <code>machine_id</code> first (409 if exists). Returns full <code>EquipmentOut</code>
GET	<code>/api/v1/equipment/</code>	List all equipment with pagination (<code>page</code> , <code>page_size</code>) and optional filters (<code>plant_name</code> , <code>machine_type</code> , <code>machine_criticality</code>)
GET	<code>/api/v1/equipment/{id}</code>	Get single equipment by UUID. Returns full <code>EquipmentOut</code> with sensors
PUT	<code>/api/v1/equipment/{id}</code>	Full update — replaces all provided fields
PATCH	<code>/api/v1/equipment/{id}</code>	Partial update — only updates fields present in request body
DELETE	<code>/api/v1/equipment/{id}</code>	Delete equipment and all its sensors (cascade). Returns 204 No Content
POST	<code>/api/v1/equipment/{id}/image</code>	Upload equipment image. Validates MIME type (JPEG/PNG/WebP/GIF) and file size ($\leq 10\text{MB}$). Saves to <code>uploads/{id}.ext</code>
GET	<code>/api/v1/equipment/{id}/image</code>	Serve uploaded image as <code>FileResponse</code> (direct binary download)

DELETE	/api/v1/equipment/{id}/image	Remove image file from disk and clear path in database
GET	/api/v1/equipment/{id}/sensors	List all sensors for an equipment
POST	/api/v1/equipment/{id}/sensors	Add a new sensor to an equipment
PUT	/api/v1/equipment/{id}/sensors/{sid}	Update a specific sensor
DELETE	/api/v1/equipment/{id}/sensors/{sid}	Remove a sensor
GET	/api/v1/equipment/{id}/ai-readiness	Calculate and return AI readiness score (0-100%) with breakdown

3.9 backend/app/routers/lookups.py

Role: Provides static reference data (dropdown values) to the frontend. Instead of hard-coding lists in both frontend and backend, they live here and the frontend could fetch them dynamically. Currently the frontend uses local constants for speed, but the API is available for future dynamic use.

GET /api/v1/lookups/	Returns all lookup categories as a single JSON object
GET /api/v1/lookups/{name}	Returns values for one category (e.g. /lookups/machine-types)
Available lookups	machine-types (13 types), machine-criticality, drive-types, load-types, foundation-types, coupling-types, motor-pole-counts, direction-of-rotation, operating-environments (13 values), lubrication-types, sensor-types (14 types), mounting-locations (11), sensor-orientations, mounting-methods (9), sensitivity-units, sampling-rates (9 presets + Custom), frequency-ranges (6 presets + Custom), asset-status

3.10 alembic/ — Database Migrations

Role: Alembic manages database schema versioning — like git for the database. Instead of manually running SQL, developers write migration scripts that Alembic applies in order.

alembic.ini	Configuration: tells Alembic where to find migration scripts and how to connect (reads from .env)
alembic/env.py	Migration environment — imports all models so Alembic can auto-detect schema changes
alembic/script.py.mako	Template used when generating new migration files
versions/001_initial_schema.py	First (and currently only) migration. Creates both tables with all columns, indexes, unique constraints, and foreign keys
upgrade()	Creates equipment_masters and sensor_configurations tables. Adds UNIQUE on machine_id. Adds indexes on plant_name and machine_type
downgrade()	Drops both tables (in reverse order to respect foreign key constraint)
Command to run	cd backend && alembic upgrade head

4. Frontend — File-by-File Documentation

4.1 src/main.tsx & index.html

index.html is the single HTML file Vite serves. It has a `<div id="root">` and a script tag loading `/src/main.tsx`.

main.tsx is the JavaScript entry point. It mounts the React tree into the DOM.

ReactDOM.createRoot()	Creates a React 18 concurrent root at <code>#root</code> div
QueryClientProvider	Provides TanStack React Query's query client to the entire app — enables all <code>useQuery/useMutation</code> hooks
BrowserRouter	Enables HTML5 history-based routing (React Router v6)
App	The root component, which contains all routes

4.2 src/App.tsx

Role: The root React component. Defines the route tree and wraps everything in the **ToastProvider** context (so toast notifications work anywhere in the app).

<code>/ → Dashboard</code>	The equipment list page (default route)
<code>/equipment/new → NewEquipmentPage</code>	The empty 6-tab form for adding a new machine
<code>/equipment/:id/edit → EditEquipmentPage</code>	The pre-filled 6-tab form for editing an existing machine
<code>* → Navigate('/')</code>	Any unknown URL redirects to the dashboard
ToastProvider	Wraps all routes so any component can call <code>showToast()</code> to display notifications

4.3 src/api/client.ts

Role: Creates and exports a pre-configured Axios instance as the single HTTP client used by all API functions. This is the bridge between React and FastAPI.

baseUrl	Reads from <code>VITE_API_BASE_URL</code> environment variable. Falls back to <code>http://localhost:8000</code>
Content-Type header	Sets 'application/json' as default. Overridden to 'multipart/form-data' for image uploads
Why a shared instance?	All API calls share the same base URL and headers. Adding auth tokens later only requires changing this one file

4.4 src/api/equipment.ts

Role: Contains every API call function used by the frontend. Each function maps to one backend endpoint. These functions are called from React hooks (useQuery, useMutation).

createEquipment(data)	POST /api/v1/equipment/ — sends full form data, receives EquipmentOut
updateEquipment(id, data)	PATCH /api/v1/equipment/{id} — sends partial data
getEquipment(id)	GET /api/v1/equipment/{id} — fetches single equipment for edit form
listEquipment(params)	GET /api/v1/equipment/ — paginated list with optional filters
deleteEquipment(id)	DELETE /api/v1/equipment/{id}
uploadEquipmentImage(id, file)	POST /api/v1/equipment/{id}/image — builds FormData, sends as multipart
addSensor / updateSensor / deleteSensor	Sensor CRUD — POST/PUT/DELETE on /sensors sub-resource
getAIReadiness(id)	GET /api/v1/equipment/{id}/ai-readiness
getLookup(name)	GET /api/v1/lookups/{name} — fetches one dropdown list
getAllLookups()	GET /api/v1/lookups/ — fetches all dropdown data at once

4.5 src/types/equipment.ts

Role: The type system for the entire frontend. Defines Zod validation schemas (which also generate TypeScript types via inference), interface definitions for API responses, and UI constants like criticality color maps.

sensorSchema (Zod)	Validates a single sensor entry. Required: sensor_type, mounting_location, orientation. All others optional with type coercion (z.coerce.number())
equipmentSchema (Zod)	Full equipment form schema. All fields are optional at the schema level (backend handles required), but Zod coerces number strings to numbers
EquipmentFormData (type)	TypeScript type inferred from equipmentSchema — used everywhere in form components
EquipmentListItem	Interface for dashboard table rows — lightweight, no sensor data
EquipmentOut	Full API response type extending EquipmentFormData with id, image path, timestamps
AIReadiness	Interface for readiness score response: score_percent + 5 booleans
PaginatedEquipment	Wraps list response with pagination metadata
CRITICALITY_COLORS	Maps 'Low'→green, 'Medium'→yellow, 'High'→red, 'Critical'→purple CSS classes for badges
CRITICALITY_DOT	Maps criticality to dot color classes for the colored bullet in list

4.6 src/pages/Dashboard.tsx

Role: The main landing page. Shows the equipment inventory as a searchable, filterable, paginated table with summary statistics at the top.

<code>useQuery(['equipment', ...])</code>	Fetches equipment list from GET <code>/api/v1/equipment/</code> . Re-fetches when page, <code>filterType</code> , or <code>filterCriticality</code> change
Stats Bar (4 cards)	Counts: Total (<code>data.total</code>), Critical (filter items), High (filter items), Active (filter items). Rendered from a map over a config array
Search box	Client-side filter — searches <code>machine_name</code> , <code>machine_id</code> , <code>plant_name</code> in the already-loaded page of results
Type / Criticality dropdowns	Server-side filters — changing them triggers a new API call with updated query params
Equipment Table	7 columns: Machine (icon + name + manufacturer), ID (monospace badge), Type, Plant/Area, Criticality (colored badge), Status (colored pill), Actions (edit/delete icons)
Delete flow	Calls <code>window.confirm()</code> , then <code>deleteEquipment()</code> via <code>useMutation</code> . On success, invalidates <code>['equipment']</code> query to refresh the list
Edit button	Navigates to <code>/equipment/{id}/edit</code>
Pagination	Previous/Next buttons. Only shown when <code>total > 20</code> . Page state is local React state
Loading/Error states	Spinner while loading, red error message if backend unreachable

4.7 `src/pages/EquipmentMaster.tsx`

Role: Two exported page components — **NewEquipmentPage** and **EditEquipmentPage**. New just renders the form with no data. Edit fetches the equipment first, transforms date strings, then passes the data as `initialData` to the form.

NewEquipmentPage	Renders with no props. Form starts empty with default values
EditEquipmentPage	Reads <code>:id</code> from URL params. Calls GET <code>/api/v1/equipment/{id}</code> via <code>useQuery</code> . Shows spinner while loading
Date transformation	API returns ISO datetime strings — splits on 'T' to extract date-only (YYYY-MM-DD) for the HTML date inputs
initialData prop	Spreads API response + overrides <code>sensors/operating_environment</code> arrays. Passes <code>editId</code> so form knows to call PATCH instead of POST

4.8 `src/components/equipment/EquipmentForm.tsx`

Role: The master form container. Manages the 6-tab navigation, the form context, and the final submission logic. All tab components are children rendered here.

useForm + zodResolver	Creates the form with Zod validation. <code>mode='onChange'</code> validates as user types
FormProvider	Wraps all tabs — lets each tab component call <code>useFormContext()</code> to access form state without prop drilling
TABS array	Defines 6 tabs with <code>id</code> , <code>label</code> , <code>icon</code> . Controls navigation rendering

completedTabs state	Tracks which tabs have been visited. Completed tabs show a blue checkmark badge
pendingImage state	Stores the selected File object. Image is uploaded AFTER equipment is created (to get the equipment id first)
goToTab()	Updates completedTabs when moving forward, updates activeTab
onSubmit()	1) Calls createEquipment() or updateEquipment(). 2) If pendingImage exists, calls uploadEquipmentImage() with the returned equipment.id. 3) Shows toast. 4) Navigates to / after 1.2s
Tab rendering	Uses conditional rendering (activeTab === N) — only one tab is in the DOM at a time
Footer navigation	Back/Next buttons + progress dots. Tab 6 shows 'Save & Finish' submit button instead of Next

4.9 src/components/equipment/tabs/ — All 6 Tabs

Tab 1: BasicDetailsTab.tsx

Three-column grid layout. Column 1: Location Hierarchy (plant, area, line). Column 2: Machine Identification (name, ID, type, criticality). Column 3: Manufacturer & Model + Equipment Image upload.

Image upload logic: Uses a hidden `<input type="file">` with a ref. When a file is selected, it calls `URL.createObjectURL()` for instant preview. The actual File object is passed up to `EquipmentForm` via `onImageSelect` prop. Upload to backend happens only after the equipment record is saved (to have an ID).

Tab 2: MechanicalDetailsTab.tsx

Single section card with a 4-column grid. Fields: Rated Power (kW), Rated RPM, Drive Type (dropdown), Load Type (dropdown), Foundation Type (dropdown), Coupling Details (dropdown). All dropdowns use `Controller + SelectInput` for consistent styling.

Tab 3: RotatingComponentsTab.tsx

Two section cards: Bearing Details and Rotating Component Specifications.

Conditional field rendering — watches `machine_type` and `drive_type`:

- Motor / Generator / DG Set → shows Motor Pole Count
- Fan / Blower → shows Number of Fan Blades
- Pump → shows Number of Pump Vanes
- Gearbox machine type OR Gear Drive → shows Gearbox Ratio + Gear Teeth
- Otherwise → shows an info message explaining the fields appear based on selection

Tab 4: OperatingProcessTab.tsx

Two section cards: Process & Operating Details, and Lubrication & Maintenance.

Operating speeds and load ranges use a custom **RangeInput** component with Min/Max fields. Operating Environment uses a **MultiSelect** component (13 options, multi-choice). Maintenance section has date pickers and a textarea for notes.

Tab 5: SensorsOrientationTab.tsx

Two-panel layout: left panel (2/3 width) = sensor table, right panel (1/3 width) = SVG diagram.

Default sensor rows: 6 pre-defined rows for standard motor-pump measurement points: DE Horizontal, DE Vertical, DE Axial, NDE Horizontal, NDE Vertical, NDE Axial. Each row has editable Mounting Location and Orientation dropdowns (local state, not submitted to backend via form).

Additional sensors: Uses React Hook Form's useFieldArray to manage a dynamic list of sensor objects that ARE included in the form submission. Each sensor has 7+ fields.

Custom values: When sampling_rate = 'Custom' or frequency_range = 'Custom', extra integer input fields appear inline for the custom Hz values.

SensorMountingDiagram: Receives a map of location→orientation. Arrows in the SVG turn blue (active) when a location has an orientation configured.

Tab 6: ReviewSaveTab.tsx

Two-column layout: left (2/3) = full form data summary, right (1/3) = AI Readiness widget.

The left column shows every filled-in field grouped in expandable sections. Null/empty fields are skipped (ReviewRow returns null if no value).

The right column shows:

- Circular SVG gauge (0–100%) with color coding
- 5-item checklist with green/grey icons
- Asset Configuration section (status dropdown + 3 boolean checkboxes)

4.10 src/components/equipment/SensorMountingDiagram.tsx

Role: A pure SVG visualization component. Shows a schematic cross-section of a motor-pump train with arrows indicating measurement directions.

Layout: NDE block (blue, left) → Shaft (center bar) → DE block (yellow, right). Six arrows: Vertical↑ on NDE, Vertical↑ on DE, Axial← on NDE, Axial→ on DE, Horizontal on NDE (left side), Horizontal↓ on DE (bottom).

Color logic: getColor(location) — if orientations[location] has a value, returns blue (#2563eb); otherwise returns grey (#d1d5db). This turns arrows blue when the corresponding row in the sensor table has a selection.

4.11 src/components/ui/ — Reusable UI Components

FormField.tsx	Label wrapper with error display. Also exports: TextInput (styled input), SelectInput (styled select), TextareaInput, RangeInput (min-to-max input pair with unit label)
MultiSelect.tsx	Custom dropdown for selecting multiple values from a list (used for operating environment). Shows selected items as pills, remaining as checkboxes in dropdown

SectionCard.tsx	White card with border, shadow, title row with icon, and a content area. Used as a consistent section wrapper throughout all tabs
Toast.tsx	Toast notification system. ToastProvider creates a React Context with showToast(). Renders absolute-positioned toast messages (success=green, error=red). Auto-dismisses after 3.5s

4.12 src/lib/utils.ts

Exports a single `cn()` utility function that merges Tailwind class names. Uses **clsx** (conditional class composition) and **tailwind-merge** (resolves conflicting Tailwind classes, e.g. `p-2 + p-4 → p-4`). Used throughout the component tree for conditional styling.

5. Database — Schema & Design

5.1 equipment_masters Table

Primary table. Every registered machine is one row.

Column	Type	Constraint	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique equipment identifier
plant_name	VARCHAR(255)	NOT NULL	Factory / plant name
area	VARCHAR(255)	NOT NULL	Zone within the plant
line	VARCHAR(255)	NOT NULL	Production line
machine_name	VARCHAR(255)	NOT NULL	Descriptive name
machine_id	VARCHAR(100)	NOT NULL, UNIQUE, INDEX	Asset code
machine_type	VARCHAR(50)	NOT NULL, INDEX	Equipment category
machine_criticality	VARCHAR(20)	NOT NULL	Low/Medium/High/Critical
manufacturer	VARCHAR(255)	nullable	OEM name
model	VARCHAR(255)	nullable	Model number
serial_number	VARCHAR(100)	nullable	Serial number
rated_power_kw	NUMERIC(10,2)	nullable	Nameplate power in kW
rated_rpm	INTEGER	nullable	Nameplate speed
drive_type	VARCHAR(50)	nullable	Direct/Belt/Gear/VFD...
load_type	VARCHAR(50)	nullable	Constant/Variable...
foundation_type	VARCHAR(50)	nullable	Concrete/Skid...
coupling_details	VARCHAR(50)	nullable	Flexible/Gear/Jaw...
bearing_details	TEXT	nullable	Free text bearing description
bearing_number_de	VARCHAR(100)	nullable	Drive end bearing part#
bearing_number_nde	VARCHAR(100)	nullable	Non-drive end bearing part#
gearbox_ratio	NUMERIC(8,3)	nullable	e.g. 4.250
gear_teeth	INTEGER	nullable	Number of gear teeth
motor_pole_count	INTEGER	nullable	2/4/6/8/10/12
fan_blades	INTEGER	nullable	Number of fan blades
pump_vanes	INTEGER	nullable	Number of impeller vanes
direction_of_rotation	VARCHAR(30)	nullable	CW/CCW/Bidirectional
operating_speed_min	INTEGER	nullable	Min actual RPM
operating_speed_max	INTEGER	nullable	Max actual RPM
load_range_min	NUMERIC(5,2)	nullable	Min load %

load_range_max	NUMERIC(5,2)	nullable	Max load %
normal_operating_load	NUMERIC(5,2)	nullable	Typical load %
process_details	TEXT	nullable	Process description
operating_environment	TEXT[]	nullable	PostgreSQL array of tags
lubrication_type	VARCHAR(50)	nullable	Grease/Oil Bath...
installation_date	DATE	nullable	Commission date
last_maintenance_date	DATE	nullable	Last service date
maintenance_notes	TEXT	nullable	Free text notes
equipment_image_path	VARCHAR(500)	nullable	Relative file path
asset_status	VARCHAR(30)	DEFAULT 'Active'	Operational state
machine_train_configured	BOOLEAN	DEFAULT false	AI readiness flag
bearing_database_mapped	BOOLEAN	DEFAULT false	AI readiness flag
operating_mode_configured	BOOLEAN	DEFAULT false	AI readiness flag
created_at	TIMESTAMPTZ	DEFAULT now()	Creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Last update timestamp

5.2 sensor_configurations Table

Child table. Many sensors can belong to one equipment (one-to-many).

Column	Type	Constraint	Description
id	UUID	PK	Sensor identifier
equipment_id	UUID	FK → equipment_masters.id ON DELETE CASCADE, INDEX	Owner equipment
sensor_type	VARCHAR(50)	NOT NULL	IEPE Accelerometer, Velocity Sensor, etc.
mounting_location	VARCHAR(100)	NOT NULL	Bearing Housing DE/NDE, Motor, etc.
orientation	VARCHAR(30)	NOT NULL	Horizontal/Vertical/Axial
mounting_method	VARCHAR(50)	nullable	Stud/Magnetic/Adhesive...
sensitivity	NUMERIC(10,4)	nullable	e.g. 100.0000 mV/g
sensitivity_unit	VARCHAR(20)	nullable	mV/g, mV/mm/s, etc.
sampling_rate	VARCHAR(20)	nullable	e.g. '4096 Hz' or 'Custom'
sampling_rate_custom	INTEGER	nullable	Hz value if Custom
frequency_range	VARCHAR(20)	nullable	e.g. '0-5000 Hz' or 'Custom'
frequency_range_custom_min	INTEGER	nullable	Min Hz if Custom

frequency_range_custom_max	INTEGER	nullable	Max Hz if Custom
is_active	BOOLEAN	DEFAULT true	Active/inactive flag
created_at	TIMESTAMPTZ	DEFAULT now()	Creation timestamp

5.3 Database Design Decisions

UUID primary keys	UUIDs (not integers) prevent enumeration attacks and allow safe distributed key generation. PostgreSQL's gen_random_uuid() generates them server-side
CASCADE delete	sensor_configurations.equipment_id has ON DELETE CASCADE — deleting equipment automatically deletes all its sensors, maintaining referential integrity
ARRAY column	operating_environment uses PostgreSQL's native ARRAY(TEXT) type — avoids creating a junction table for a simple multi-select field
Indexes	machine_id has UNIQUE index for fast duplicate checks. plant_name and machine_type have non-unique indexes for filter queries
Numeric precision	rated_power_kw: NUMERIC(10,2), gearbox_ratio: NUMERIC(8,3), sensitivity: NUMERIC(10,4) — precise decimal storage without floating-point errors
Timestamps	Both tables use TIMESTAMPTZ (timezone-aware). equipment_masters.updated_at auto-updates via SQLAlchemy onupdate
No soft delete	Currently uses hard delete. CASCADE ensures no orphan sensor rows

6. API Reference — All Endpoints

Base URL: **http://localhost:8000** | All equipment endpoints: **/api/v1/equipment** | Lookups: **/api/v1/lookups**

6.1 Equipment Endpoints

Method	Endpoint	Description
POST	/api/v1/equipment/	Create equipment. Body: EquipmentCreate JSON. Returns: EquipmentOut (201)
GET	/api/v1/equipment/	List equipment. Params: page, page_size, plant_name, machine_type, machine_criticality. Returns: PaginatedEquipment
GET	/api/v1/equipment/{id}	Get single equipment with all sensors. Returns: EquipmentOut
PUT	/api/v1/equipment/{id}	Full update. Body: EquipmentUpdate JSON. Returns: EquipmentOut
PATCH	/api/v1/equipment/{id}	Partial update. Body: partial EquipmentUpdate. Returns: EquipmentOut
DELETE	/api/v1/equipment/{id}	Delete equipment + sensors. Returns: 204 No Content

6.2 Image Endpoints

Method	Endpoint	Description
POST	/api/v1/equipment/{id}/image	Upload image. Body: multipart/form-data, field 'file'. Max 10MB. Allowed: JPEG, PNG, WebP, GIF
GET	/api/v1/equipment/{id}/image	Download/view image. Returns: binary FileResponse
DELETE	/api/v1/equipment/{id}/image	Remove image file and clear DB path. Returns: 204

6.3 Sensor Endpoints

Method	Endpoint	Description
GET	/api/v1/equipment/{id}/sensors	List all sensors for equipment
POST	/api/v1/equipment/{id}/sensors	Add sensor. Body: SensorConfigCreate
PUT	/api/v1/equipment/{id}/sensors/{sid}	Update sensor. Body: SensorConfigUpdate
DELETE	/api/v1/equipment/{id}/sensors/{sid}	Delete sensor

6.4 AI Readiness & Lookup Endpoints

Method	Endpoint	Description
--------	----------	-------------

GET	/api/v1/equipment/{id}/ai-readiness	Returns AIReadinessOut: score_percent (0-100) + 5 boolean checks
GET	/api/v1/lookups/	Returns all lookup categories as single JSON object
GET	/api/v1/lookups/{name}	Returns values array for one lookup category
GET	/health	Health check. Returns: {status: 'ok'}

7. Frontend ↔ Backend Connection Map

Every API call that the frontend makes, where it's made from, what triggers it, and what API function/endpoint it hits.

Trigger / Component	API Function	HTTP Call	Purpose
Dashboard mounts	listEquipment()	GET /api/v1/equipment/?page=1&page_size=20	Initial equipment list load
Filter dropdown change	listEquipment({machine_type})	GET /api/v1/equipment/?machine_type=Pump	Server-side filter
Pagination Next/Prev	listEquipment({page:2})	GET /api/v1/equipment/?page=2	Load next page
Delete icon click	deleteEquipment(id)	DELETE /api/v1/equipment/{id}	Remove equipment
+ Add Equipment btn	(React Router navigate)	/equipment/new route	Navigate to form
Edit icon click	(React Router navigate)	/equipment/{id}/edit route	Navigate to edit form
EditEquipmentPage mounts	getEquipment(id)	GET /api/v1/equipment/{id}	Load existing data into form
Form Tab 6: Save & Finish (new)	createEquipment(data)	POST /api/v1/equipment/	Create new equipment record
Form Tab 6: Save & Finish (edit)	updateEquipment(id, data)	PATCH /api/v1/equipment/{id}	Update existing record
After save, if image selected	uploadEquipmentImage(id, file)	POST /api/v1/equipment/{id}/image	Upload equipment photo
AI Readiness widget (Tab 6)	(computed locally)	No API call — computed from form state	Live readiness score

7.1 How CORS Works Here

The browser enforces the Same-Origin Policy — it blocks requests from one origin (localhost:5173) to another (localhost:8000) unless the server explicitly allows it.

FastAPI's **CORSMiddleware** sends these response headers:

- **Access-Control-Allow-Origin:** <http://localhost:5173>
- **Access-Control-Allow-Methods:** *
- **Access-Control-Allow-Headers:** *

The Axios client on the frontend does NOT need to do anything special — the browser reads these headers and allows the response to reach JavaScript code.

8. Dashboard Image — What You See & Why

The screenshot shows the Dashboard page after one equipment entry (a Pump) has been created. Here is a precise technical explanation of every visible element.

8.1 Header Bar

Blue icon (Activity)	Lucide-react 'Activity' icon. Rendered in a blue-600 rounded-xl div
'AI Vibration Intelligence Platform'	h1 with font-bold text-gray-900. Hardcoded in Dashboard.tsx line 61
'Equipment Master Data — Stage 1'	Subtitle paragraph text-xs text-gray-500. Describes the current module
'+ Add Equipment' button	Blue button that calls navigate('/equipment/new'). Only shown in header. Uses Plus icon from lucide-react

8.2 Stats Cards (4 Cards)

Four cards in a responsive grid (2-column on mobile, 4-column on desktop). Each card shows a Cpu icon in a colored background circle, a bold number, and a label.

'1 — Total Equipment'	data.total from PaginatedEquipment response = 1. This is the server-side total count
'0 — Critical'	data.items.filter(i => i.machine_criticality === 'Critical').length = 0. Filters current page client-side
'0 — High'	data.items.filter(i => i.machine_criticality === 'High').length = 0
'1 — Active'	data.items.filter(i => i.asset_status === 'Active').length = 1. The one equipment has status='Active'
Colors	Total=blue, Critical=purple, High=red, Active=green — defined in the stat config array in Dashboard.tsx

8.3 Search & Filter Bar

Search input	Controlled input, value=search state. Filters client-side over machine_name, machine_id, plant_name of current page results
'All Types' dropdown	Select with options from MACHINE_TYPES array. On change sets filterType state → triggers new API call
'All Criticality' dropdown	Select with Low/Medium/High/Critical. Sets filterCriticality → triggers new API call

8.4 Equipment Table Row

The single row visible in the screenshot corresponds to the one saved equipment record:

Machine column (blue icon + '—' + 'Pump')	Icon: Cpu in bg-blue-100 div. '—' is machine_name (either empty or set to dash). 'Pump' shows below as manufacturer — but wait: looking at the screenshot, the machine name appears to be blank/dash and 'Pump' is in the Type column
---	---

ID column (grey badge)	machine_id rendered in font-mono bg-gray-100 badge. Appears blank/loading
Type column: 'Pump'	machine_type = 'Pump' from the saved record
Plant/Area column	plant_name / area from the record (appears empty in screenshot)
Criticality: 'Medium' (yellow badge)	machine_criticality = 'Medium'. Badge uses CRITICALITY_COLORS['Medium'] = 'bg-yellow-100 text-yellow-800 border-yellow-200'. The yellow dot uses CRITICALITY_DOT['Medium'] = 'bg-yellow-500'
Status: 'Active' (green pill)	asset_status = 'Active'. Conditional class: bg-green-100 text-green-700
Edit icon (pencil)	Edit2 icon from lucide-react. Navigates to /equipment/{id}/edit
Delete icon (trash)	Trash2 icon. Opens window.confirm() then calls deleteEquipment(id)

8.5 Why These Values?

The data visible in the dashboard comes from the database record created when someone submitted the 6-tab Equipment Form with:

- **machine_type = 'Pump'** (selected in Tab 1, BasicDetailsTab dropdown)
- **machine_criticality = 'Medium'** (selected in Tab 1 criticality dropdown)
- **asset_status = 'Active'** (default value set in Tab 6, ReviewSaveTab)
- machine_name and machine_id appear empty/dash — the user may not have filled those fields, or they submitted with defaults

The backend stored this via **POST /api/v1/equipment/**, SQLAlchemy wrote it to PostgreSQL, and the dashboard fetches it via **GET /api/v1/equipment/** on page load using React Query.

9. AI Readiness Score — Calculation & Logic

The AI Readiness Score quantifies how well an equipment record is prepared for AI-based vibration analysis. A score of 100% means the AI engine has all the data it needs to: calculate fault frequencies, detect anomalies, and generate alerts.

9.1 The 5 Checks

1. Machine Train Configured	equipment.machine_train_configured == True. A boolean flag the user explicitly checks in Tab 6. Indicates: machine chain (motor + coupling + pump) has been defined
2. Asset Status Set	equipment.asset_status is not None and not empty string. Almost always True since default = 'Active'
3. Sensor Coverage	len(equipment.sensors) > 0. At least one sensor must be configured in Tab 5. This is the most commonly missing check
4. Bearing Database Mapped	equipment.bearing_database_mapped == True. A boolean flag the user checks. Indicates: bearing catalogue numbers (DE + NDE) have been entered in Tab 3
5. Operating Mode Configured	equipment.operating_speed_min is not None AND equipment.operating_speed_max is not None. The speed range in Tab 4 must be filled

9.2 Score Calculation (Backend — crud/equipment.py)

The `compute_ai_readiness()` function:

```
checks = {
    'machine_train_configured': equipment.machine_train_configured or False,
    'asset_status_set': equipment.asset_status not in (None, ''),
    'sensor_coverage': len(equipment.sensors) > 0,
    'bearing_database_mapped': equipment.bearing_database_mapped or False,
    'operating_mode_configured': (
        equipment.operating_speed_min is not None and
        equipment.operating_speed_max is not None
    ),
}
score = int(sum(checks.values()) / len(checks) * 100)
# sum(checks.values()) = number of True checks (0-5)
# / len(checks) = / 5
# * 100 = percentage
# int() truncates (not rounds): 3/5*100 = 60, 2/5*100 = 40
```

0 checks pass	score = 0%
1 check passes	score = 20%
2 checks pass	score = 40%
3 checks pass	score = 60%
4 checks pass	score = 80%

5 checks pass

score = 100%

9.3 Frontend Display (ReviewSaveTab.tsx)

The frontend computes the same score **locally from form state** using identical logic — so users see a live score as they fill in the form without making an API call:

```
const score = Math.round(
  (checks.filter(c => c.value).length / checks.length) * 100
);
```

Note: The frontend uses `Math.round()` while the backend uses `int()` (truncation). For multiples of 20 these give the same result.

Gauge color coding:

- score $\geq 80\%$ → green (#16a34a)
- score $\geq 60\%$ → blue (#2563eb)
- score $\geq 40\%$ → amber (#f59e0b)
- score $< 40\%$ → red (#ef4444)

Gauge SVG math: The circle has $r=40$, so circumference = $2\pi \times 40 \approx 251.2$. `strokeDasharray = [(score/100) × 251.2, 251.2]`. The SVG is rotated -90° so the arc starts at the top (12 o'clock position).

Text below gauge:

- 100% → "Fully AI-Ready!"
- $\geq 60\%$ → "Good Coverage"
- $< 60\%$ → "Needs More Data"

10. Full Request/Response Workflow

10.1 Creating New Equipment (Happy Path)

Step	Where	What Happens
1. User clicks '+ Add Equipment'	Dashboard.tsx	navigate('/equipment/new') — React Router renders NewEquipmentPage
2. Form initializes	EquipmentForm.tsx	useForm({resolver: zodResolver(equipmentSchema)}) sets up form state with default values. FormProvider wraps all tabs
3. User fills Tab 1	BasicDetailsTab.tsx	useFormContext() reads/writes form state. Image file stored in pendingImage state (not form)
4. User clicks 'Next'	EquipmentForm.tsx	goToTab(2) called. Tab 1 added to completedTabs set. activeTab = 2
5. User fills Tabs 2-5	Each tab component	All controlled by React Hook Form via useFormContext(). Sensor arrays managed by useFieldArray
6. Tab 6 auto-renders	ReviewSaveTab.tsx	watch() reads all form values. AI score computed locally from checks. User sees full summary
7. User clicks 'Save & Finish'	EquipmentForm.tsx	handleSubmit(onSubmit) runs. Zod validates all fields. If valid, onSubmit() called
8. createEquipment() called	api/equipment.ts	axios.post('/api/v1/equipment/', data). Full form JSON sent
9. FastAPI receives POST	routers/equipment.py	Pydantic validates body as EquipmentCreate. Calls crud.create_equipment(db, data)
10. DB write	crud/equipment.py	Equipment ORM object created, db.flush() to get UUID. SensorConfiguration objects created. db.commit()
11. Response returned	FastAPI → axios → React	EquipmentOut JSON returned (HTTP 201). axios resolves the promise
12. Image upload (if selected)	EquipmentForm.tsx	uploadEquipmentImage(equipment.id, pendingImage). POST /api/v1/equipment/{id}/image with FormData
13. Backend saves image	routers/equipment.py	Validates MIME + size. Saves to uploads/{id}.ext. Updates equipment_image_path in DB
14. Toast shown	Toast.tsx	showToast('Equipment saved!', 'success') — green notification appears for 3.5s
15. Navigate	EquipmentForm.tsx	setTimeout(() => navigate('/'), 1200) — returns to dashboard after 1.2s
16. Dashboard refreshes	Dashboard.tsx	React Query automatically has stale ['equipment'] query → refetches → new row appears

10.2 Editing Equipment

1. Edit icon clicked	navigate('/equipment/{id}/edit') — React Router loads EditEquipmentPage
2. useQuery fires	GET /api/v1/equipment/{id} — fetches full equipment with sensors
3. Data transformation	Date strings split on 'T' to get YYYY-MM-DD for HTML date inputs

4. Form pre-filled	initialData passed to EquipmentForm → useForm defaultValues populated
5. User edits any field	Form state updated reactively by React Hook Form
6. Save & Finish	updateEquipment(editId, data) → PATCH /api/v1/equipment/{id} with changed fields
7. SQLAlchemy UPDATE	model_dump(exclude_unset=True) ensures only sent fields are updated in DB

10.3 Deleting Equipment

1. User clicks trash icon	handleDelete(id, name) called
2. Confirmation dialog	window.confirm('Delete X? This cannot be undone.') — browser native dialog
3. If confirmed	deleteMutation.mutate(id) → DELETE /api/v1/equipment/{id}
4. Backend processes	crud.delete_equipment() loads equipment, calls db.delete() — CASCADE removes sensors
5. On success	queryClient.invalidateQueries(['equipment']) — React Query marks list as stale, re-fetches
6. Toast shown	showToast('Equipment deleted.', 'success')

11. Deployment & Infrastructure

11.1 Docker Compose (docker-compose.yml)

Two Docker services are defined. They are started with `docker-compose up -d` from the project root.

postgres service	Image: postgres:16. Container: vibration_platform_db. Exposes port 5433 (host) → 5432 (container). Uses named volume postgres_data for persistence. Reads POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB from .env
pgadmin service	Image: dpage/pgadmin4. Container: vibration_platform_pgadmin. Exposes port 5050 → 80. Web UI for browsing the database. Depends on postgres starting first
Health check	postgres service has a pg_isready health check (10s interval, 5 retries) so dependent services wait for DB to be ready
Named volumes	postgres_data and pgadmin_data persist data across container restarts

11.2 Backend Startup

1. Create .env	Copy .env.example, fill in DATABASE_URL pointing to localhost:5433
2. Create venv	<code>python -m venv .venv && .venv\Scripts\activate</code>
3. Install deps	<code>pip install -r requirements.txt</code>
4. Run migrations	<code>alembic upgrade head</code> — creates tables in PostgreSQL
5. Start server	<code>uvicorn app.main:app --reload --port 8000</code>
Swagger UI	<code>http://localhost:8000/docs</code> — auto-generated interactive API documentation

11.3 Frontend Startup

1. Install deps	<code>cd frontend && npm install</code>
2. Start dev server	<code>npm run dev</code> — Vite starts on <code>http://localhost:5173</code> with HMR
3. Build for prod	<code>npm run build</code> — TypeScript compile + Vite bundle → dist/ folder
VITE_API_BASE_URL	Set in frontend/.env to override the backend URL (default: <code>http://localhost:8000</code>)

11.4 Port Summary

5173	Vite dev server — React frontend
8000	Uvicorn — FastAPI backend
5433	PostgreSQL (Docker, mapped from container's 5432)
5050	pgAdmin web UI (Docker)

End of Documentation

AI Vibration Intelligence Platform — Stage 1 Equipment Master Data